UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

FACULTAD DE INGENIERÍA DE SISTEMAS E INFORMATICA

E.P DE INGENIERÍA DE SOFTWARE

Análisis y diseño de Algoritmos

Laboratorio 02

DOCENTE

Chàvez Soto, Jorge Luis

ALUMNO

Bohorquez Huaringa Kevin Rhamses

2025

Ejercicio de la Subsecuencia Máxima

Suma de Subsecuencia máxima: Dados un conjunto de enteros consecutivos (posiblemente negativos) a1, a2, ...,an Encontrar el subconjunto ai,...,ak que proporcione la sumatoria máxima de los (k-i+1) números consecutivos desde i hasta k consecutivos. La suma es cero si todos los números son negativos. Encontrar un algoritmo eficiente y que determinen su función tiempo y su notación asintótica.

Algoritmo implementado en Java:

```
public class Lab02_SubcadenaMaxima {
   public static void main(String[] args) {
       int[] original = {7, -4, 10, 7, 3, -3, 4, 8, -2, -10};
       int[] ajustado = new int[original.length];

       ajustado[0] = original[0];
       for (int i = 1; i < original.length; i++) {
           ajustado[i] = ajustado[i - 1] + original[i];
       }

       int acum = 0;
       int aux = Integer.MIN_VALUE;
       int min = -1;
       int max = -1;

       for (int i = 0; i < ajustado.length; i++) {
           int suma = ajustado[i] + acum;

           if (suma < 0) {
               acum += Math.abs(suma);
               min = i;
           } else {
               if (suma > aux) {
                   aux = suma;
                   max = i;
               }
           }

       }
```

```java
        if (min == -1) min = 0;
        if (ajustado[min] < 0) min++;

        System.out.print("Arreglo: {");
        for (int i = 0; i < original.length; i++) {
            System.out.print(original[i]);
            if (i != original.length - 1) System.out.print(", ");
        }
        System.out.println("}");

        System.out.println("Índice mínimo: " + min);
        System.out.println("Índice máximo: " + max);

        int aux2 = 0;
        // Mostrar subcadena de interés
        if (min != -1 && max != -1 && min <= max) {
            System.out.print("Subcadena maxima: {");
            for (int i = min; i <= max; i++) {
                System.out.print(original[i]);
                if (i != max) System.out.print(", ");
                aux2 += original[i];
            }
            System.out.println("}");
            System.out.println("Suma de la subcadena maxima: " + aux2);
        } else {
            System.out.println("No se encontró una subcadena válida.");
        }
    }
}
```

Función tiempo y notación asintótica

Supongamos que el arreglo de entrada contiene n elementos. El algoritmo tiene los siguientes bloques:

1. **Construcción del arreglo ajustado**:
   Se recorre el arreglo una vez para calcular el acumulado en cada posición.

→ Tiempo: **O(n)**

2. **Búsqueda de la subcadena con suma máxima**:
   Se recorre nuevamente el arreglo ajustado y se realizan operaciones de suma y comparación constantes.
   → Tiempo: **O(n)**

3. **Impresión del arreglo original**:
   Se imprime cada elemento del arreglo original.
   → Tiempo: **O(n)**

4. **Impresión de la subcadena de interés**:
   Se imprime la subcadena comprendida entre los índices mínimo y máximo. En el peor caso, abarca todo el arreglo.
   → Tiempo: **O(n)**

Sumando todos los bloques, la función tiempo total es:

$$T(n)=c1 \cdot n+c2 \cdot n+c3 \cdot n+c4 \cdot n=O(n)$$

Notación asintótica

- El tiempo de ejecución está acotado superiormente por una función lineal
- También está acotado inferiormente por una función lineal, ya que requiere recorrer todo el arreglo al menos una vez
- Por lo tanto, el tiempo de ejecución está acotado tanto superior como inferiormente por la misma función lineal

  Por lo tanto: $T(n) \in \Theta(n)$